

Keecas Quarto Example

Table of contents

How to use keecas in a Quarto document	2
Basic Symbolic Math with Units	2
Pipe Commands Demonstration	2
Different LaTeX Environments	3
Cases Environment	3
Rcases Environment	3
Equation Environment	3
Gather Environment	3
Custom Environment	3
Equation With Labels	4
Label Generation	4
Check Function	6
Check Function Templates	7
Custom Templates	7
Template Configuration	8
Automatic Dependency Ordering	8
Common Pitfalls	9
Pitfall 1: LaTeX vs Plain Symbol Notation	9
Pitfall 2: Using <code>vals</code> dict in <code>pc.subs</code>	9
Summary	11
Features Covered	11
Key Patterns	11
Common Pitfalls	11
Links	11

How to use keecas in a Quarto document

This example demonstrates key features of **keecas** for symbolic and units-aware calculations in a Quarto document.

Note

If the rendering engine is **KaTeX** (i.e. Jupyter notebook), the label command `\label{}` will result in a **ParseError**. The latex code will work when rendering by `quarto render`, but since you may want to see the result before rendering the document, an option to disable the `\label{}` command is provided.

When rendering with `quarto` you can have two config:

- if `qmd` files, then everything is fine
- if `ipynb` files, then pass the `--execute` flag to `quarto render` to rerender the notebook

Basic Symbolic Math with Units

Define symbols and calculate basic engineering quantities:

$$F_d = 93 \text{ kN} \quad \text{applied force} \quad (1)$$

$$A_{load} = 50 \text{ cm}^2 \quad \text{cross-sectional area} \quad (2)$$

$$\sigma_{Sd} = \frac{F_d}{A_{load}} = 18.60 \text{ MPa} \quad \text{normal stress} \quad (3)$$

Pipe Commands Demonstration

Using pipe operators for functional composition:

$$x = 3 \text{ m} \quad (4)$$

$$y = 4 \text{ m} \quad (5)$$

$$d = x^2 + y^2 = 25.0 \text{ m}^2 \quad (6)$$

Different LaTeX Environments

Different environment are available for display with `align` being the default. * version of the environment are also available.

$$E_{steel} = 200 \text{ GPa} \quad \text{steel elastic modulus} \quad (7)$$

$$E_{concrete} = 30 \text{ GPa} \quad \text{concrete elastic modulus} \quad (8)$$

Cases Environment

$$\left\{ \begin{array}{ll} E_{steel} = 200 \text{ GPa} & \text{steel elastic modulus} \\ E_{concrete} = 30 \text{ GPa} & \text{concrete elastic modulus} \end{array} \right. \quad (9)$$

Rcases Environment

$$\left. \begin{array}{ll} E_{steel} = 200 \text{ GPa} & \text{steel elastic modulus} \\ E_{concrete} = 30 \text{ GPa} & \text{concrete elastic modulus} \end{array} \right\} \quad (10)$$

Equation Environment

$$E_{steel} = 200 \text{ GPa} \text{steel elastic modulus} \quad E_{concrete} = 30 \text{ GPa} \text{concrete elastic modulus} \quad (11)$$

Gather Environment

Custom Environment

A custom environment can be defined. Here we define a custom environment with raw LaTeX blocks encapsulation:

```
```{=latex}
\begin{align}
a &= 5{\backslash,}\backslashtext{m} & \backslashquad\backslashtext{length a} & \backslash\backslash[8pt]
b &= 12{\backslash,}\backslashtext{m} & \backslashquad\backslashtext{length b} & \backslash\backslash[8pt]
c &= a + b & \backslashquad\backslashtext{total length c} &
\end{align}
```
```

$$a = 5 \text{ m} \quad \text{length a} \quad (12)$$

$$b = 12 \text{ m} \quad \text{length b} \quad (13)$$

$$c = a + b \quad \text{total length c} \quad (14)$$

$$a = 5 \text{ m} \quad \text{length a}$$

$$b = 12 \text{ m} \quad \text{length b}$$

$$c = a + b \quad \text{total length c}$$

Equation With Labels

Calculate beam deflection with cross-references:

$$\delta = \frac{5 q L^4}{384 E I} = 15.95 \text{ mm} \quad (15)$$

Note

labels emitted by `show_eqn` are latex labels, therefore they need to be referenced with `\eqref{}` or `\ref{}` latex commands:

- `@eq-QUARTO_EXAMPLE-delta` will not work -> `(?@eq-QUARTO_EXAMPLE-delta)`
- `\eqref{eq-QUARTO_EXAMPLE-2kp5fgk9}` will work -> (15)

The deflection calculated in (15) shows acceptable values for serviceability.

Label Generation

Various methods for generating labels for equation cross-references:

Manual Label Assignment

When manually assigning labels, they should be LaTeX-safe:

$$J = 123 \text{ cm}^4 \tag{16}$$

$$E = 456 \text{ MPa} \tag{17}$$

$$a = b + \frac{c_0}{2} \tag{18}$$

Controlled Automatic Generation

Generate labels with `eq-` prefix automatically:

Non hashed Labels

$$J = 123 \text{ cm}^4 \tag{19}$$

$$E = 456 \text{ MPa} \tag{20}$$

$$a = b + \frac{c_0}{2} \tag{21}$$

hashed Labels (recommended)

$$J = 123 \text{ cm}^4 \tag{22}$$

$$E = 456 \text{ MPa} \tag{23}$$

$$a = b + \frac{c_0}{2} \tag{24}$$

With an helper function, the generated labels saved to a dict can be easily referenced in the quarto document:

For example J is defined at [\(22\)](#), while E is defined at [\(23\)](#)

Full Automatic Generation

A callable can be passed to `label` argument of `show_eqn`. For each key, all the key and values will be passed to the callable.

$$J = 123 \text{ cm}^4 \quad \text{moment of inertia} \quad (25)$$

$$E = 456 \text{ MPa} \quad \text{modulus of elasticity} \quad (26)$$

$$a = b + \frac{c_0}{2} \quad \text{an expression} \quad (27)$$

Note

When using this method, the interpolated label will change if any of the values in the key-row of the Dataframe change. For a more stable approach, either pass an already interpolated dict of labels as string (recommended), or use a custom callable with a more resilient logic (e.g. use only the element of the list that are less likely to change).

Check Function

Using the `check` function for design checks:

$$\sigma_{Sd} = 20 \text{ MPa} \quad (28)$$

$$\tau_{Sd} = 15 \text{ MPa} \quad (29)$$

$$\sigma_{Rd} = 250 \text{ MPa} \quad (30)$$

$$\tau_{Rd} = 75 \text{ MPa} \quad (31)$$

$$\frac{\sigma_{Sd}}{\sigma_{Rd}} = 0.080 \quad [\leq 1.0 \quad \text{VERIFICATO}] \quad (32)$$

$$\frac{\tau_{Sd}}{\tau_{Rd}} = 0.200 \quad [\leq 1.0 \quad \text{VERIFICATO}] \quad (33)$$

The type of comparison in the `check` function can be specified, as well as the value to be checked against:

$$a = 3 \quad \quad \quad [> 1 \quad \text{NON VERIFICATO}] \quad \quad (34)$$

$$b = 4 \quad \quad \quad [> 2 \quad \text{VERIFICATO}] \quad \quad (35)$$

$$c = 5 \quad \quad \quad [\geq 3 \quad \text{NON VERIFICATO}] \quad \quad (36)$$

$$d = 6 \quad \quad \quad [\geq 4 \quad \text{VERIFICATO}] \quad \quad (37)$$

$$f = 7 \quad \quad \quad [\neq 5 \quad \text{NON VERIFICATO}] \quad \quad (38)$$

Check Function Templates

The `check` function supports customizable templates for different visual styles:

Default : $[\leq 1.0 \quad \text{VERIFICATO}]$ $[> 1.0 \quad \text{NON VERIFICATO}]$

Boxed : $\leq 1.0 \checkmark \text{ VERIFICATO}$ $> 1.0 \times \text{NON VERIFICATO}$

Minimal : $\leq 1.0 \checkmark$ $> 1.0 \times$

$$a = 3 \quad \quad \quad (39)$$

$$b = 4 \quad \quad \quad (40)$$

$$d = \frac{\sqrt{a^2 + b^2}}{c} \quad \quad \quad = 0.417 \quad \quad (41)$$

$$c = a \, b \quad \quad \quad = 12.000 \quad \quad (42)$$

Custom Templates

You can also use completely custom templates:

$$\text{Pass : } \leq 1.0 \text{ PASS} \quad \quad (43)$$

$$\text{Fail : } > 1.0 \text{ FAIL} \quad \quad (44)$$

Template Configuration

Templates can also be configured globally via TOML config files:

```
[check_templates.template_sets.minimal]
success = '${symbol}{rhs} \,\textcolor{green}{\checkmark}}$'
failure = '${symbol}{rhs} \,\textcolor{red}{\times}}$'
```

This allows you to set project-wide or user-wide styling for all check functions.

Automatic Dependency Ordering

Using `pc.subs` will automatically order the substitutions pairs topologically, so you don't have to order them yourself (default `sympy` behavior). For example, let's define some mappings in any order:

$$\begin{cases} x = y^2 \\ y = 1 + a \\ a = 3 \end{cases} \quad (45)$$

If we use the default behavior of `sympy`'s `subs` method (unsorted substitutions) we get:

$$\begin{cases} x = (1 + a)^2 & \text{partially evaluated} \\ y = 4 \\ a = 3 \end{cases} \quad (46)$$

While the default behavior of `pc.subs` is to sort the substitutions:

$$\begin{cases} x = 16 & \text{fully evaluated} \\ y = 4 \\ a = 3 \end{cases} \quad (47)$$

Note

`pc.subs` automatically converts objects to `sympy` expressions before substitution. In the example above, `a` maps to the Python `int` value 3. When piping `int` through `pc.subs`, it works seamlessly. If you used `sympy`'s `subs()` method directly, you'd get an error since `int` objects don't have a `subs()` method. You'd need to write `S(rhs).subs(...)` to explicitly convert first.

Common Pitfalls

Important Patterns to Know

Understanding these common pitfalls will save you debugging time and ensure correct calculations.

Pitfall 1: LaTeX vs Plain Symbol Notation

Symbols are compared by their string representation, so LaTeX and plain notation create different objects:

σ_{Sd}

`$\sigma_{Sd}$`

Are they equal? `False`

Pitfall 2: Using `vals` dict in `pc.subs`

While maintaining a global `vals` dict for evaluated results can be useful, passing it to `pc.subs` will overwrite expressions with already evaluated values, preventing recalculation when parameters change.

$$x = 2 \tag{48}$$

$$y = x + 1 \qquad \qquad \qquad = 3.0 \tag{49}$$

$$z = y^2 \qquad \qquad \qquad = 9.0 \tag{50}$$

Now update the parameter and see the problem:

$$x = 3 \tag{51}$$

$$y = 4 \tag{52}$$

$$z = 9.0 \tag{53}$$

 Tip

Recommendation: Only pass `vals` to `pc.subs` if you're certain no dependent expressions need updating.

Summary

This example demonstrated keecas capabilities for engineering calculations in Quarto documents.

Features Covered

- **Symbolic math with units** - SymPy expressions with Pint units
- **Pipe commands** - Functional composition (`pc.parse_expr`, `pc.subs`, `pc.convert_to`, `pc.N`)
- **LaTeX environments** - align, cases, equation, and custom environments
- **Label generation** - Manual, automatic, and unique hash-based labels
- **Verification** - `check()` function with customizable templates
- **Dependency ordering** - Automatic topological sorting with `pc.subs`

Key Patterns

```
# Cell-local dicts (live in one cell)
_p = {F_d: 10*u.kN}                    # Parameters
_e = {sigma: "F_d/A_load" | pc.parse_expr} # Expressions
_v = {k: v | pc.subs(_e|_p) | pc.N for k,v in _e.items()} # Values

# Global persistence (across cells)
params.update(_p)
eqn.update(_e)
```

Common Pitfalls

Warning

1. Use LaTeX notation: `symbols(r"\sigma_{Sd}")` not `symbols("sigma_Sd")`
2. Avoid passing `vals` to `pc.subs` - it prevents recalculation

Links

- [pypi](#)
- [Repository](#)

- [Documentation](#)
- [Issues](#)